

Leaderless Replication

1. What is Leaderless Replication?

Leaderless replication means there is **no primary/leader node**.

All replicas are equal, and any node can handle:

None

Read requests

Write requests

Clients or a coordinator node communicate with multiple replicas.

2. Basic Idea

Instead of:

None

Write → Primary → Followers

We use:

None

Write → Replica A

Write → Replica B

Write → Replica C

Any node can receive requests.

3. Why Use It?

Used to remove single-leader bottleneck and improve:

None

High availability

Fault tolerance

Multi-region scalability

No single point of failure

Horizontal write scaling

4. Popular Databases Using Leaderless Model

- Apache Cassandra
- Amazon DynamoDB (inspired by Dynamo principles)
- Riak
- Project Voldemort

5. How Writes Work

Suppose replication factor:

None

N = 3 replicas

Write request sent to multiple nodes.

Example:

None

Client writes balance=500

Replica A ✓

Replica B ✓

Replica C timeout

Write may still succeed if quorum reached.

6. Quorum Concept (Very Important)

A **quorum** = minimum number of successful nodes required.

Let:

None

$N = \text{total replicas}$

$W = \text{write quorum}$

$R = \text{read quorum}$

If:

None

$R + W > N$

Then reads overlap writes, helping consistency.

7. Example

None

$N = 3$

$W = 2$

$R = 2$

Because:

None

$2 + 2 > 3$

At least one node overlaps, so latest value likely found.

8. Read vs Write Tradeoff

Fast Writes

Use:

None

W low

R high

Write quickly, reads check more replicas.

Fast Reads

Use:

None

R low

W high

Writes slower, reads faster.

9. Eventual Consistency

If quorum is relaxed:

None

$W = 1$

$R = 1$

Then stale reads possible.

Replicas sync later.

This is:

None

Eventually Consistent

10. Conflict Problem

Two users update same key on different nodes simultaneously.

None

Node A = price 100

Node B = price 120

Need conflict resolution.

11. Common Conflict Handling

A. Last Write Wins

Timestamp newest wins.

B. Vector Clocks

Tracks causality/history.

C. Application Merge

Business-specific merge logic.

D. Manual Resolution

Rare but possible.

12. Deletes Use Tombstones

Instead of immediate delete:

None

DELETE user123

System stores delete marker:

None

Tombstone

Why?

Because old replicas may still have original row. Tombstone propagates deletion safely.

Used in Apache Cassandra.

13. Read Repair

If read detects stale replica:

None

Replica A = old

Replica B = new

Replica C = new

System can auto-update stale node.

Called:

None

Read Repair

14. Hinted Handoff

If one replica is down during write:

None

Node C unavailable

Another node temporarily stores missed write and delivers later.

Called:

None

Hinted Handoff

15. Advantages

None

No single leader failure

High write availability

Geo-distributed friendly

Scales horizontally

Good for always-on systems

16. Disadvantages

None

Conflict resolution hard

Complex reads

Stale data possible

Deletes are expensive (tombstones)

Operational complexity

17. Best Use Cases

Use leaderless for:

- Social feeds
- Messaging metadata
- Product catalogs
- Session data
- IoT writes
- Large-scale event systems

18. Avoid For

Avoid when strict consistency required:

- Banking ledger
- Stock trading
- Inventory with exact counts
- Strong transaction systems

Use SQL / consensus systems there.

19. Comparison

Model	Writes	Reads	Complexity	Consistency
Single Leader	One node	Scalable	Low	Stronger
Multi Leader	Many leaders	Good	High	Conflict-prone

Leaderless	Any node	Any node	High	Tunable
------------	----------	----------	------	---------

20. Interview HLD Points

If using leaderless replication, mention:

None

Replication factor N

Read quorum R

Write quorum W

Conflict resolution

Repair strategy

Tombstones

Cross-region replication

21. One-Line Summary

Leaderless replication removes the primary node and allows any replica to serve reads/writes, using quorum and repair mechanisms to balance availability, scalability, and consistency.